



Administrator Linux. Professional

Web: протоколы



Проверить, идет ли запись

Меня хорошо видно && слышно?



Преподаватель



Лавлинский Николай

Технический директор «Метод Лаб»

Более 20 лет в веб-разработке

Преподавал в ВУЗе более 10 лет
Более 7 лет в онлайн-образовании

Специализация: оптимизация производительности, ускорение сайтов и веб-приложений

https://t.me/methodlab_tg

<https://www.methodlab.ru/>

<https://rutube.ru/channel/24617406/>

<https://www.youtube.com/c/NickLavlinsky>

https://www.youtube.com/@site_support

<https://vkvideo.ru/@methodlab>

<https://habr.com/ru/users/Nickmob/>

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Телеграм-чате



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

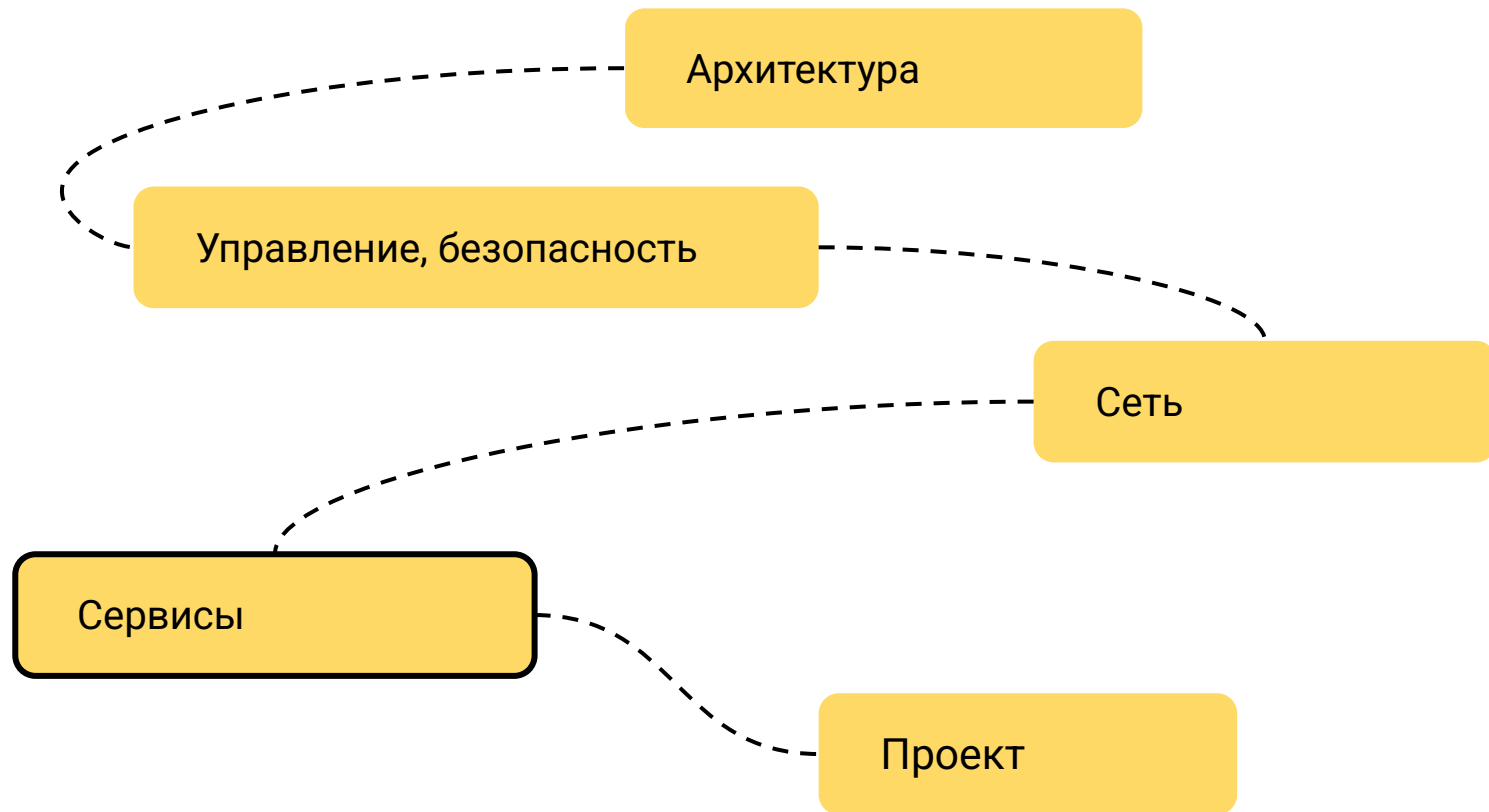


Документ



Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара



История Интернета

Ранние версии HTTP

HTTP/2 и HTTP/3

TLS и SSL

Let's encrypt и сертификаты

Цели вебинара

После занятия вы сможете

1. Понимать различие версий HTTP
2. Использовать HTTPS
3. Работать с TLS-сертификатами

Смысл

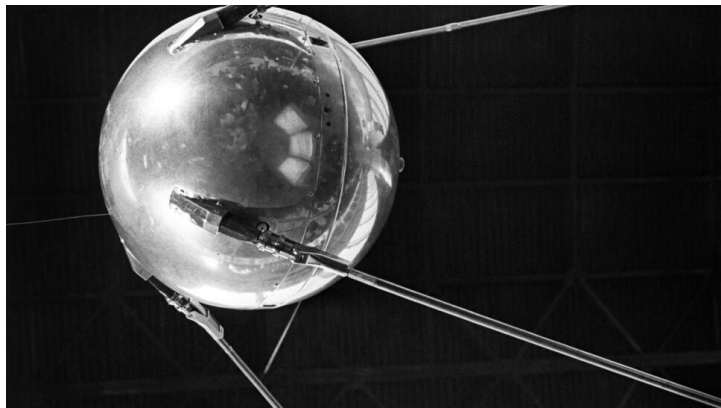
Зачем вам это уметь

1. Настраивать веб-приложения
2. Решать проблемы взаимодействия по HTTP
3. Настраивать защищённые сервисы с помощью TLS

История развития Интернета

Краткая история появления Интернета

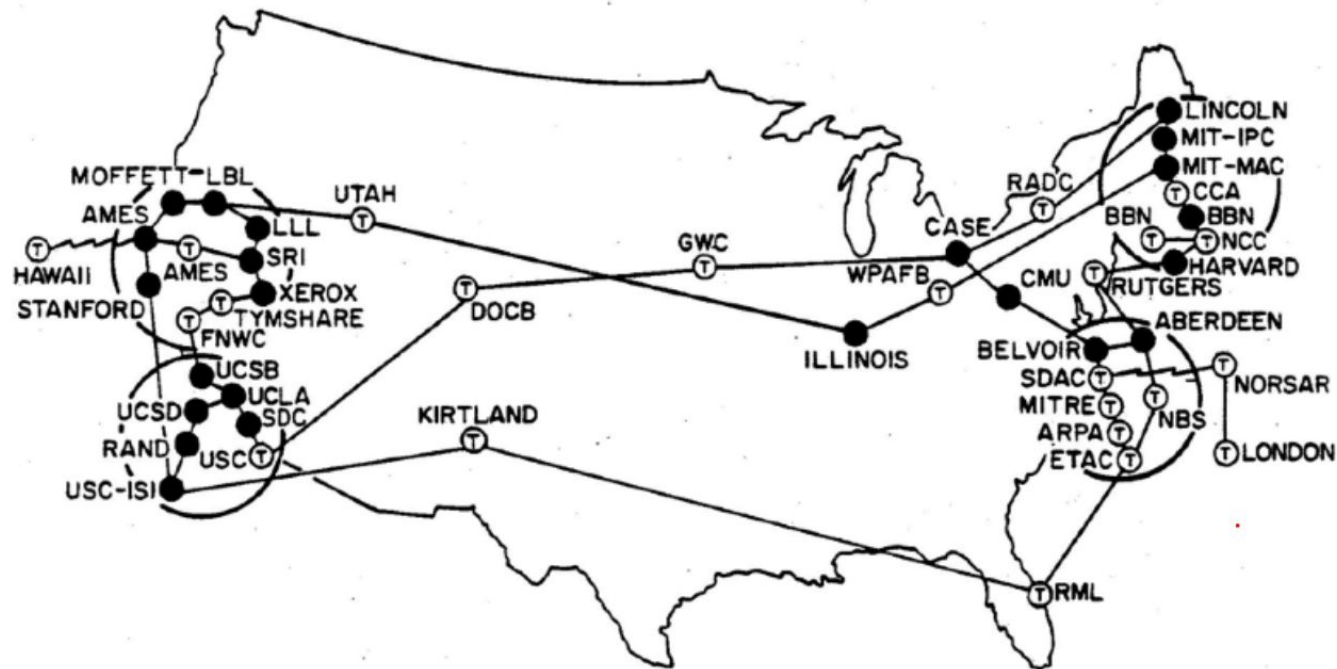
- 1957: запуск Спутника-1
- 1958: создание NASA
- 1958: идея “советского Интернета” Китова А. И.
- 1969: появление ARPANET (Advanced Research Projects Agency NETwork), Пол Бэран
- 1983: Переход ARPANET на TCP/IPv4
- 1986: ARPANET преобразуется в Internet, разработана система DNS
- 1989–1991: Появление WWW (World Wide Web), Тим Бернерс-Ли



<https://theconversation.com/how-the-internet-was-born-a-stuttered-hello-67903>
https://ru.wikipedia.org/wiki/Китов,_Анатолий_Иванович

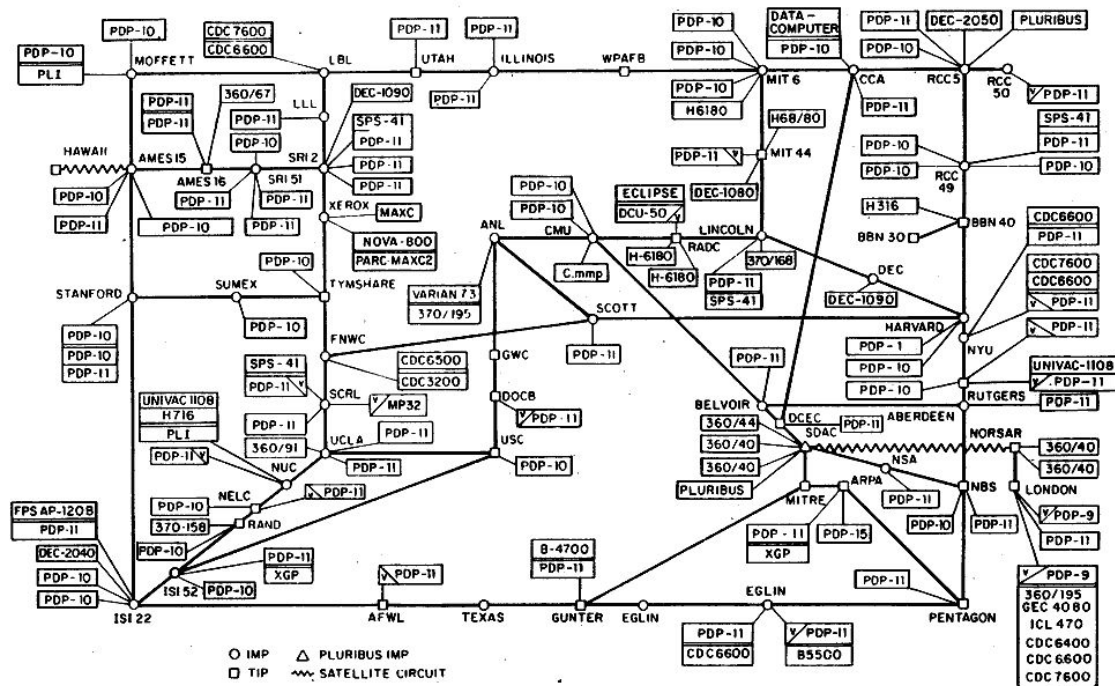


Узлы ARPANET в начале 1970-х



Узлы ARPANET в 1977 году

ARPANET LOGICAL MAP, MARCH 1977



(PLEASE NOTE THAT WHILE THIS MAP SHOWS THE HOST POPULATION OF THE NETWORK ACCORDING TO THE BEST INFORMATION OBTAINABLE, NO CLAIM CAN BE MADE FOR ITS ACCURACY)

NAMES SHOWN ARE IMP NAMES, NOT (NECESSARILY) HOST NAMES

Адресация в WWW

Краткая история появления Интернета

- **HTTP** (HyperText Transport Protocol) — протокол передачи данных
- **HTML** (HyperText Markup Language) — язык разметки
- **URI** (Uniform Resource Identifier) — унифицированный идентификатор ресурса.

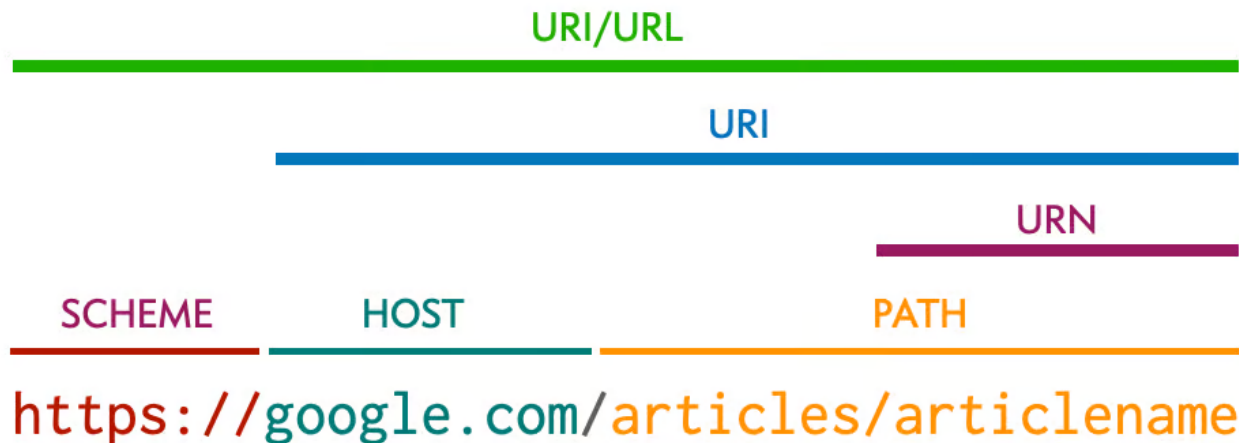
Позволяет идентифицировать физический ресурс. Ранее URI строго делились на URL и URN. Сейчас см. [ссылку](#). Пример: `https://habr.com/img/favicon-16.png`

- **URL** (Uniform Resource Locator) — помогает найти какой-либо ресурс
- **URN** (Uniform Resource Name) — единообразное название ресурса. Может по одному только названию дать вам ресурс (абстрактный или физический). Пример:
`urn:isbn:540609601X`

https://en.wikipedia.org/wiki/Uniform_Resource_Identifier
<https://www.w3.org/TR/uri-clarification/>



Соотношение URI, URL и их частей



Первые версии HTTP



HTTP 0.9

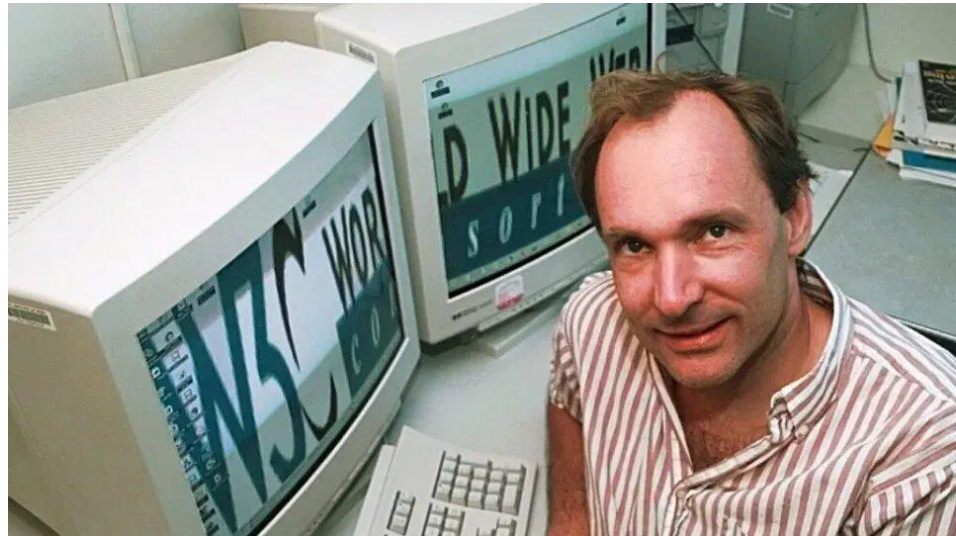
- Первая версия (1991)
- Однострочный протокол
- Нет заголовков
- Нет кодов ответа
- На фото: Тим Бернерс-Ли (Tim Berners-Lee, CERN)

```
GET /test.html
```

```
<html>
```

```
  A very simple HTML page
```

```
</html>
```



HTTP 1.0

- Указание версии
- Новые методы (POST, HEAD)
- Коды ответа (status code)
- Заголовки запроса и ответа
- Разделение типов контента (заголовок Content-Type)

```
GET /test.html HTTP/1.0
```

```
User-Agent: NCSA_Mosaic/2.0 (Windows 3.1)
```

```
200 OK
```

```
Date: Tue, 15 Nov 1994 08:12:31 GMT
```

```
Server: CERN/3.0 libwww/2.17
```

```
Content-Type: text/html
```

```
<HTML>
```

```
A page with an image
```

```
<IMG SRC="/myimage.gif">
```

```
</HTML>
```

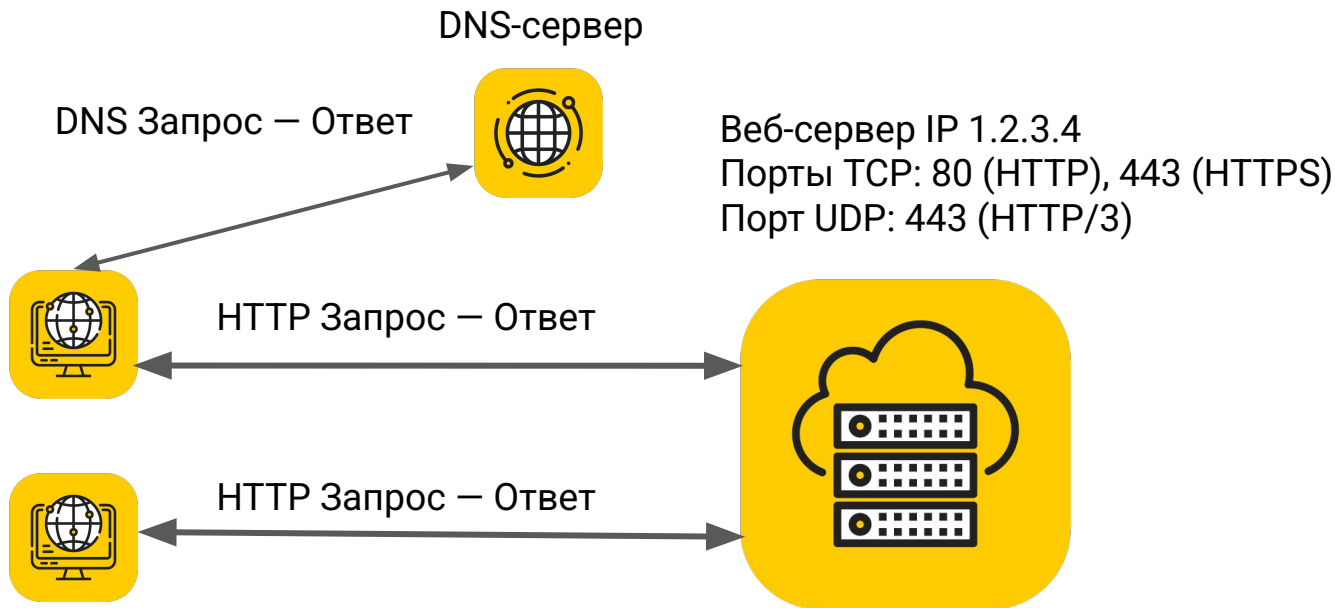


HTTP 1.1

- Стандарт (1997)
- Переиспользование соединений
- Контроль кэширования
- Заголовок Host для разделения по доменам
- Конвейеризация запросов (pipelining) — не работает
- Согласование контента (content negotiation)
- Сжатие (transfer-encoding)
- Новые методы: PUT, PATCH, DELETE, CONNECT, TRACE, OPTIONS



Базовая архитектура веб-приложения



HTTP/2

Недостатки HTTP 1.1

- Нет рабочего мультиплексирования запросов
- Необходимость создания множества соединений для браузеров
- Задержки для создания соединений (TCP + TLS)
- Отсутствие приоритизации запросов
- Нет сжатия заголовков
- Необходимость доменного шардинга
- Необходимость объединения и инлайна ресурсов

SPDY — путь к HTTP/2

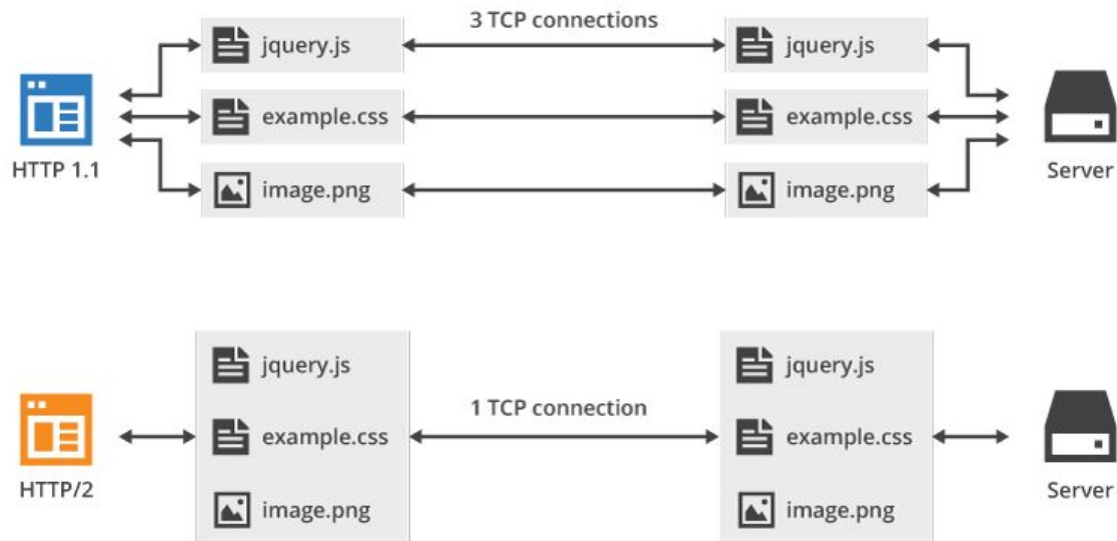
- Протокол SPDY (2009-2015)
- Разработан Google
- Версии 1, 2, 3.1
- Мультиплексирование запросов
- Приоритизация
- Поддержка большинством веб-серверов и браузеров
- Требование использования TLS (NPN)
- Server push
- Сжатие заголовков запросов и ответов

HTTP/2

- Все фичи SPDY 3.1
- Разработан IETF
- Мультиплексирование запросов
- Приоритизация
- Nginx с версии 1.9.5
- Требование использования TLS 1.2 (ALPN) и выше (H2, H2C не поддерживается)
- Поддержка большинством веб-серверов и браузеров
- Поддержка server push (deprecated)
- Поддержка сжатия заголовков запросов и ответов (HPACK)



HTTP/2 – мультиплексирование



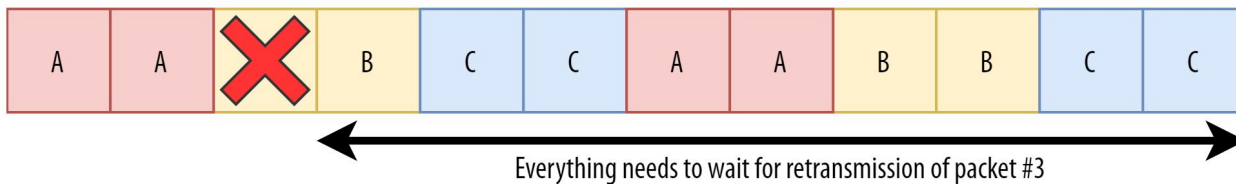
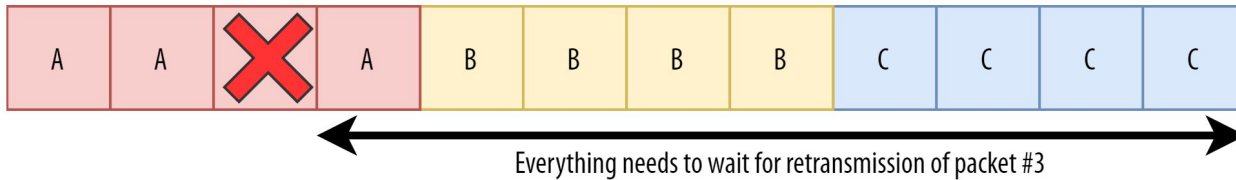
HTTP/3

Недостатки HTTP/2

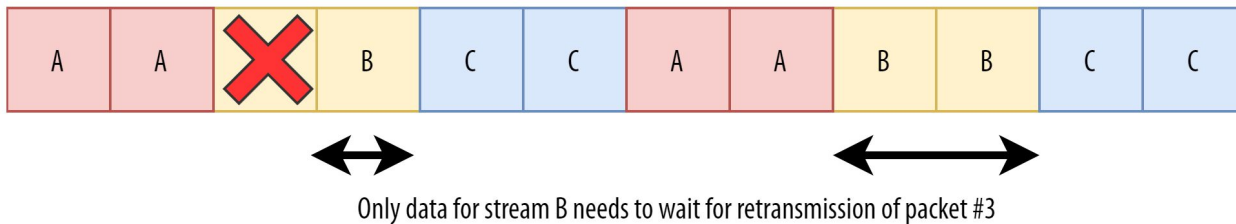
- Приоритизация работает плохо (HOL blocking — TCP)
- Необходимость задержек при подключении TCP + TLS
- Высокая чувствительность к потерям пакетов
- Зависимость от работы TCP (изменения на уровне ядра)
- Попытка решения: TCP BBR

TCP HOL blocking

HTTP/1.1 and HTTP/2 suffer from TCP Head-of-Line blocking



HTTP/3 does not suffer from Head-of-Line blocking because of QUIC



HTTP/3

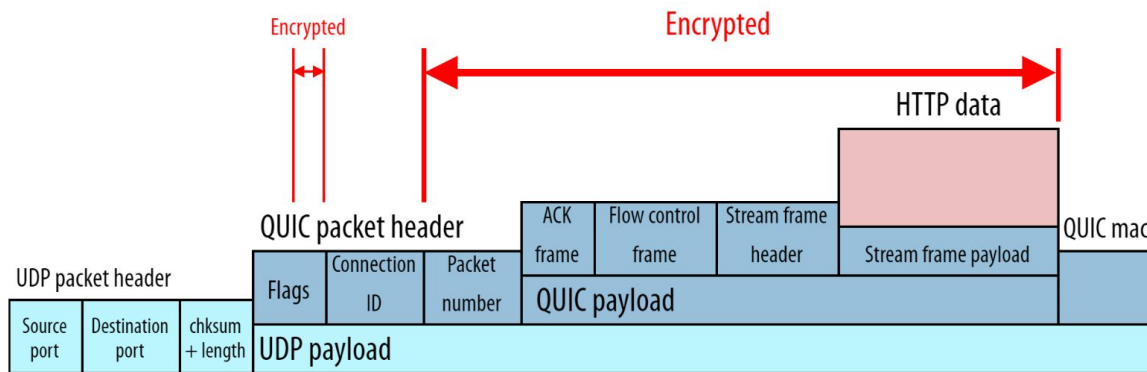
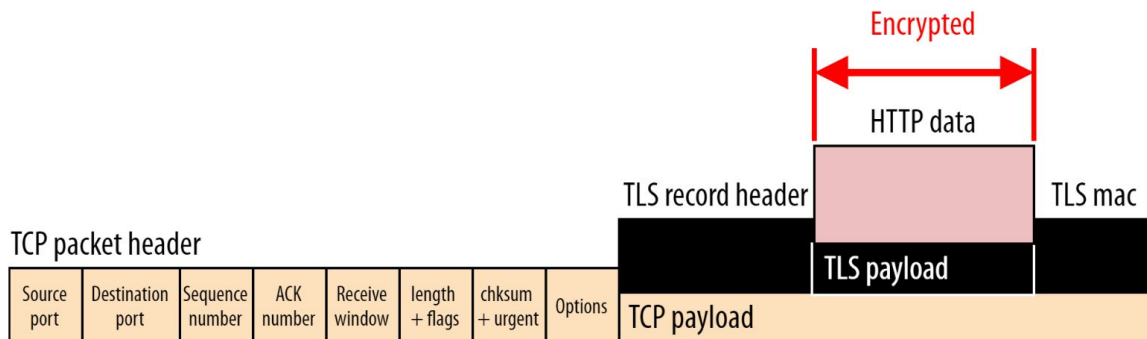
- Переход на транспорт QUIC (UDP), отказ от TCP
- Обязательный встроенный TLS
- Более полное шифрование (включая метаданные)
- Поддержка обновления протокола на уровне приложения
- Поддержка миграции соединений (при переключении сети)



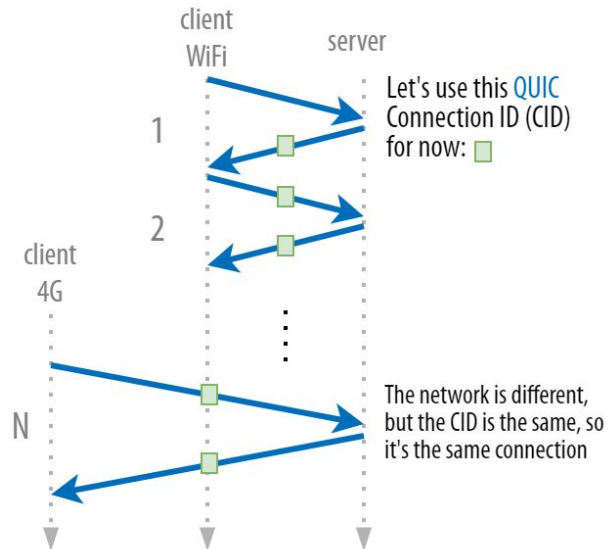
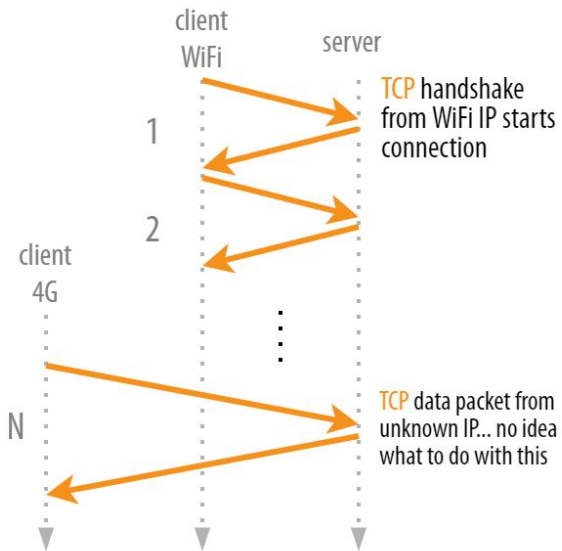
<https://www.rfc-editor.org/rfc/rfc9114>
<https://www.rfc-editor.org/rfc/rfc9000>



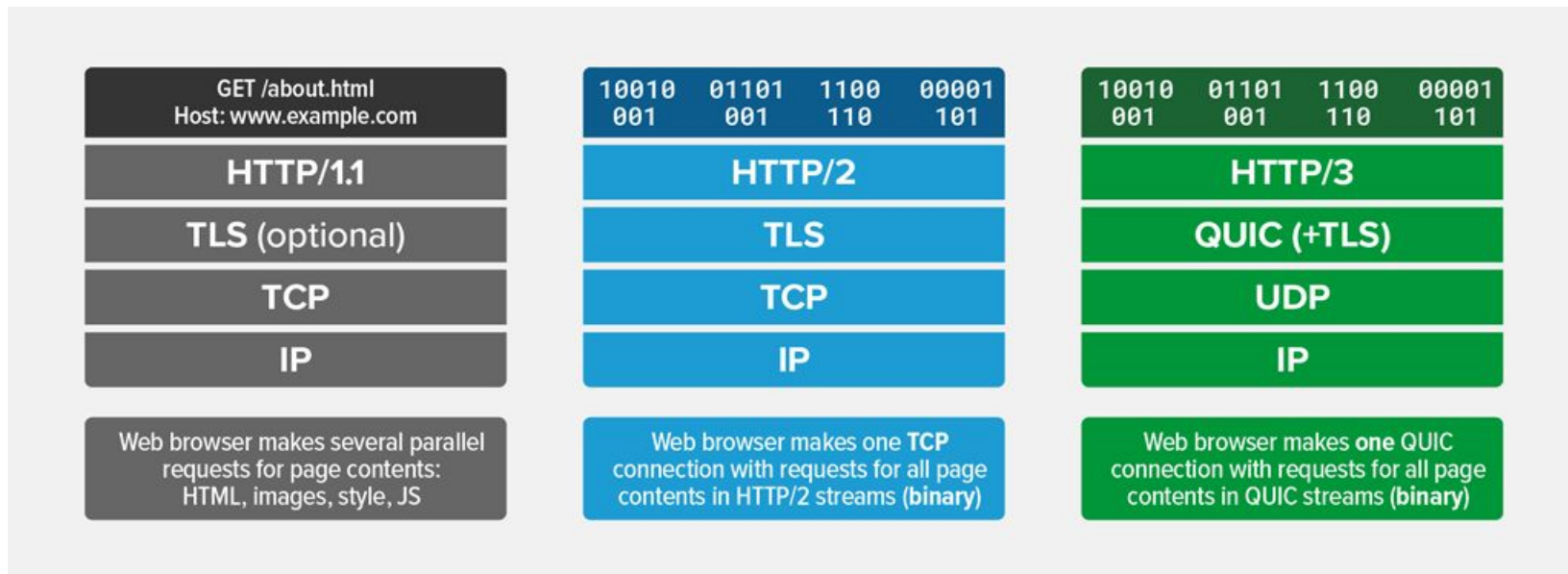
HTTP/3 — шифрование метаданных



HTTP/3 — миграция соединений



Версии HTTP – сравнение



HTTP — методы, коды, заголовки

HTTP-методы

- **GET** — получить содержимое указанного ресурса
- **HEAD** — получить только заголовки
- **POST** — отправить данные
- **DELETE** — удалить указанный ресурс
- **OPTIONS** — определить параметры сервера
- **PUT** — обновление
- **PATCH** — частичное обновление
- **TRACE** — возврат с трассировкой
- **CONNECT** — Преобразует соединение запроса в прозрачный TCP/IP-туннель

HTTP-заголовки

- Данные, уточняющие запрос или ответ
- Используются для управления запросом и или ответом
- Кэширование
- Сжатие
- Идентификация
- Аутентификация
- Безопасность
- Свойства запроса/ответа
- Keep-alive соединения
- Cookie – куки

https://en.wikipedia.org/wiki/List_of_HTTP_header_fields

Коды ответа в HTTP

- **1xx:** Informational (информационные)
- **2xx:** Success (успешно)
- **3xx:** Redirection (перенаправление)
 - 301 Moved Permanently («перемещено навсегда»)
 - 302 Found («найдено»)
- **4xx:** Client Error (ошибка клиента)
 - 403 Forbidden («запрещено (не уполномочен)»)
 - 404 Not Found («не найдено»)
- **5xx:** Server Error (ошибка сервера)
 - 500 Internal Server Error («внутренняя ошибка сервера»)
 - 502 Bad Gateway («плохой, ошибочный шлюз»)
 - 504 Gateway Timeout («шлюз не отвечает»)

[https://ru.wikipedia.org/wiki/Список кодов состояния HTTP](https://ru.wikipedia.org/wiki/Список_кодов_состояния_HTTP)



HTTPS

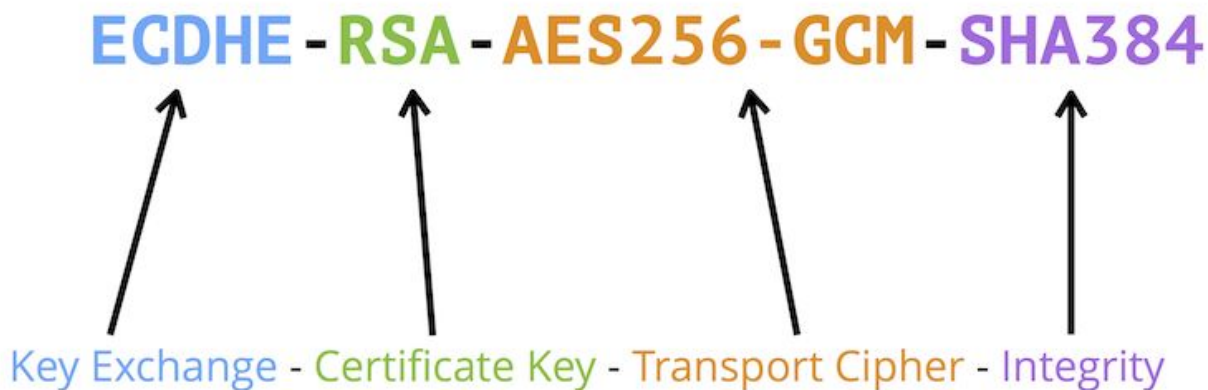
HTTPS

- HTTPS — расширение HTTP для работы с SSL/TLS
- Версии протоколов:
 - SSL 2.0 (1995–2011)
 - SSL 3.0 (1996)
 - TLS 1.0 (1999–2021)
 - TLS 1.1 (2006–2021)
 - **TLS 1.2 (2008)**
 - **TLS 1.3 (2018)**



Шифры в TLS

Cipher Suites



Шифры в TLS

Key Exchange

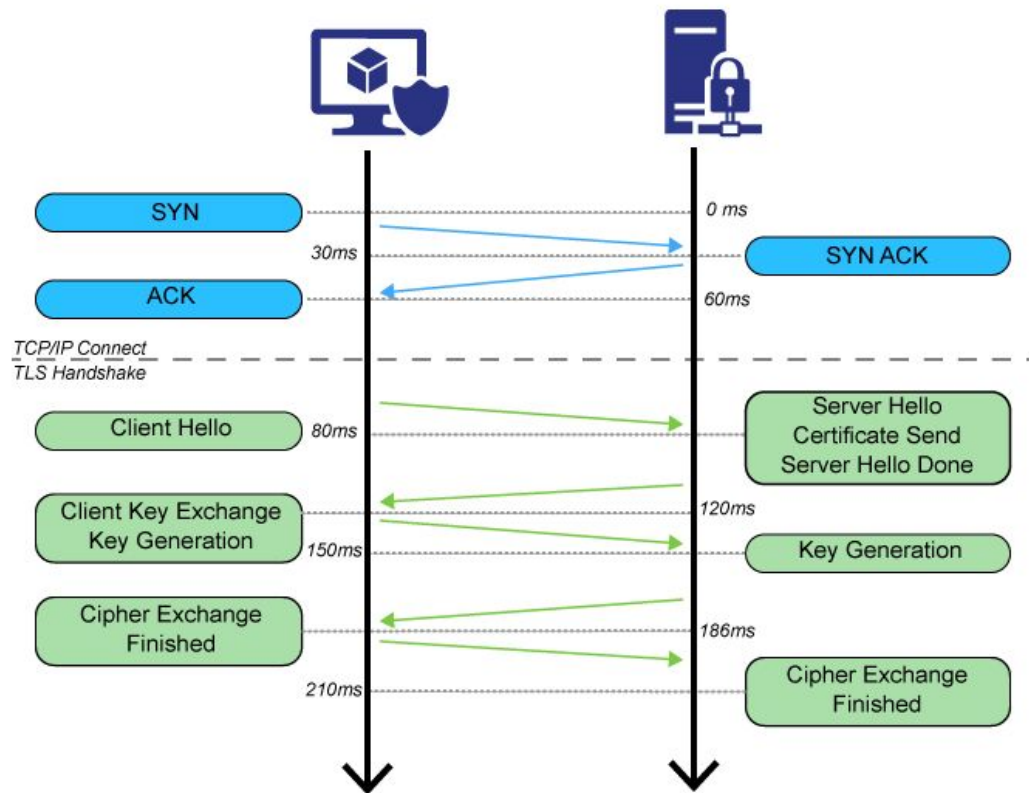
Authentication

Cipher (algorithm strength mode)

Mac or PRF

ECDHE-ECDSA-AES128-GCM-SHA256

Установка соединения TLS



PKI и создание сертификатов

Термины PKI

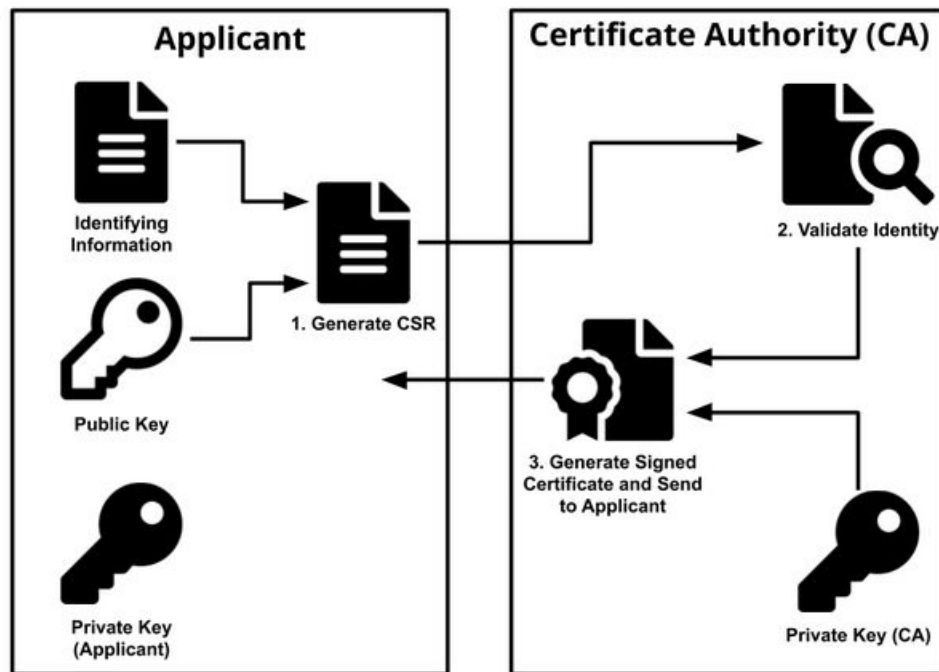
- **PKI** – Инфраструктура открытых ключей (ИОК, public key infrastructure) – набор средств, распределённых служб и компонентов, в совокупности используемых для поддержки криптозадач, на основе закрытого и открытого ключей.
- **СА** – Удостоверяющий центр, он же и центр сертификации (УЦ или СА – certificate authority) является основной структурой, формирующей цифровые сертификаты подчиненных центров сертификации и конечных пользователей.
- **Сертификат** открытого ключа (сертификат, certificate) – это данные пользователя/сервера и его открытый ключ, скреплённые электронной подписью удостоверяющего центра.
- **Ключевая пара** – это набор, состоящий из двух ключей: закрытого ключа (private key) и открытого ключа (public key), эти ключи создаются вместе.

Принципы PKI

1. Закрытый ключ (private key) известен только его владельцу.
2. Удостоверяющий центр (УЦ или CA — certificate authority) создает электронный документ — сертификат открытого ключа, таким образом удостоверяя факт того, что закрытый (секретный) ключ известен эксклюзивно владельцу этого сертификата, открытый ключ (public key) свободно передается.
3. Никто не доверяет друг другу, но все доверяют удостоверяющему центру.
4. Удостоверяющий центр подтверждает или опровергает принадлежность открытого ключа заданному лицу, которое владеет соответствующим закрытым ключом.

Самоподписанные сертификаты

Генерация сертификатов



Создание сертификата

Интерактивный режим

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \  
-keyout /etc/ssl/private/selfsigned.key \  
-out /etc/ssl/certs/selfsigned.crt
```

Сертификат для localhost

```
openssl req -x509 -out localhost.crt -keyout localhost.key \  
-newkey rsa:2048 -nodes -sha256 \  
-subj '/CN=localhost' -extensions EXT -config <( \  
    printf "[dn]\nCN=localhost\n[req]\ndistinguished_name =  
dn\n[EXT]\nsubjectAltName=DNS:localhost\nkeyUsage=digitalSignature\nextendedKeyUsage=serve  
rAuth")
```



Создание сертификата

- **req:** использовать управление запросами подписи сертификатов X.509 (CSR).
X.509 — это стандарт инфраструктуры открытых ключей, используемый SSL и TLS для управления ключами и сертификатами
- **x509:** создать самоподписанный сертификат, а не CSR
- **nodes:** пропустить опцию защиты сертификата с помощью пароля.
- **days 365:** срок, в течение которого сертификат будет считаться действительным
- **newkey rsa:2048:** сгенерировать новый сертификат и новый ключ одновременно
- **keyout:** где мы разместим создаваемый закрытый ключ
- **out:** куда поместить создаваемый сертификат

Сертификаты от Let's Encrypt

Создание бесплатного сертификата

- ACME — Automatic Certificate Management Environment: протокол для автоматизированного управления сертификатами в рамках PKI
- Let's Encrypt (LE) — Некоммерческий Центр сертификации, предоставляющий сертификаты TLS

<https://letsencrypt.org/ru/how-it-works/>



Создание сертификата LE

Автоматическая конфигурация

```
certbot --nginx -d example.com -d www.example.com
```

Только получить сертификат (WebRoot)

```
certbot certonly --webroot -w /home/www -d site.ru -d www.site.ru
```

Только получить сертификат – wildcard (DNS)

```
certbot certonly --agree-tos --manual --preferred-challenges dns \
--server https://acme-v02.api.letsencrypt.org/directory \
-d "*.site.ru" -d "site.ru"
```

<https://certbot.eff.org/>

<https://github.com/plinss/acmebot>

<https://github.com/acmesh-official/acme.sh>

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет



Что мы изучили?

Подведем итоги

1. Основные версии протокола HTTP
2. Важные аспекты работы HTTPS
3. Понятия PKI
4. Работу с сертификатами TLS

Список материалов для изучения

1. <https://pki-tutorial.readthedocs.io/en/latest/simple/index.html>
2. <https://github.com/plinss/acmebot>
3. <https://github.com/acmesh-official/acme.sh>
4. <https://theconversation.com/how-the-internet-was-born-a-stuttered-hello-67903>
5. [https://ru.wikipedia.org/wiki/Китов, Анатолий Иванович](https://ru.wikipedia.org/wiki/Китов,_Анатолий_Иванович)
6. <https://certbot.eff.org/>
7. https://en.wikipedia.org/wiki/List_of_HTTP_header_fields
8. <https://ssl-config.mozilla.org/>

Рефлексия

Цели вебинара

Проверка достижения целей

1. Понимать различие версий HTTP
2. Использовать HTTPS
3. Работать с TLS-сертификатами



Рефлексия



Что было самым полезным на занятии?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!

Приходите на следующие вебинары



Лавлинский Николай

Технический директор “Метод Лаб”

https://t.me/methodlab_tg
<https://www.methodlab.ru/>
<https://rutube.ru/channel/24617406/>
<https://www.youtube.com/c/NickLavlinsky>
https://www.youtube.com/@site_support
<https://vkvideo.ru/@methodlab>
<https://habr.com/ru/users/Nickmob/>